



OBJECT-ORIENTED PROGRAMMING

THE SOFT WARRIORS

WALKTHROUGH – SafeStore

MEMBERS:

JOEL SANCHEZ

RONALD TIPAN

ADRIAN VIZCAINO

LENIN TIPANTIZA

TEACHING: **Ing. JORGE EDISON LASCANO**

NRC: 30812

SANGOLQUÍ-ECUADOR

Artifacts reviewed:

- Product portfolio

- Use case diagrams
- Class diagrams
- Code

Review type: Walkthrough

Date: May 13, 2026

1. Findings In The Product Backlog

- Invalid menu input handling: Several modules did not properly validate invalid menu options entered by the user, causing unexpected behavior during execution.
Additional validations and error messages were implemented to improve system stability.
- Backup id duplication: The backup system initially generated duplicated backup IDs when multiple backups were created in a short period of time.
The backup identification logic was improved to reduce duplication risks.
- Notificacion trigger failure: Expiration notifications were not always triggered correctly after verifying product dates.
The notification process was corrected to ensure alerts are generated consistently.

2. Findings In Use Cases Diagram

- **Confusion over the system name**
In the VPP, SecureStore appears at the beginning, but most actors and cases use SafeStore.
Correction: The name was unified to SafeStore throughout the model.
- **The <<includes>> relationship or misapplied**
The <<includes>> relationship between Sort Products by Date and Send Notification was incorrectly applied.
Correction: The <<includes>> relationship between Send Notification has been corrected; it is now connected to Generate Expiration Alert.
- **There is no distinction between view and modify**
Several use cases allow you to view information but also modify it without separation. This is fine, but it's not visually distinguishable in the diagram.
Correction: There's no need to change anything, just keep that in mind when designing the interface.

3. Findings In Class Diagrams

SAFE CODE - FEATURE / CLASS COHERENCE REPORT

REVIEWED CLASSES

- ThermalPrinting
- WeightSales
- CreditManagement
- CashControl
- StockMinAlert
- ExpirationControl

The reviewed classes maintain coherence with the system features because the class names and methods are related to the functionality they represent.

Examples:

- ThermalPrinting → printer and ticket functions
- WeightSales → weight conversion and price calculation
- CreditManagement → payments and debt management

The project also keeps a modular structure because each feature is separated into its own class.

OBSERVATIONS

- Several classes follow a very similar structure using menus and switch-case statements.
- Most methods currently simulate actions using console messages.
- In some classes, menu interaction and system functionality are implemented together in the same file.

POSITIVE POINTS

- Descriptive class names
- Organized structure by features
- Consistency between methods and functionalities
- Similar coding style across the project

RECOMMENDATIONS

- Continue maintaining feature-based organization
- Separate menu logic from system logic in future versions
- Add more internal functionality to some methods

4. Findings In The Code

SAFE CODE - CODE QUALITY REPORT

Project: SafeStore System (SAFE CODE)

Environment: Apache NetBeans

Language: Java

Project: SafeStore System (Educational Structure)

CRITICAL ISSUES (Must Fix)

✖ 1. Compilation Error - Nested Comments

File: BackupSystem.java **Problem:** Malformed comment block prevents compilation

Quick Fix:

// Remove the `/* */` block completely or ensure proper closure

✖ 2. Language Inconsistency

Primary concern for SAFE CODE: Mix of Spanish and English

Principle: "Use one language consistently"

File	Spanish Methods	Should be English
BackupSystem	crearRespaldo, eliminarRespaldo, verRespaldos	createBackup, deleteBackup, viewBackups
CashControl	abrirCaja, cerrarCaja, calcularTotal	openCashDrawer, closeCashDrawer, calculateTotal
ComboSystem	crearCombo, activarCombo, desactivarCombo	createCombo, activateCombo, deactivateCombo
CreditManagement	agregarDeuda, realizarPago, bloquearCuenta	addDebt, makePayment, blockAccount
ExpirationControl	generarAlerta, enviarNotificacion	generateAlert, sendNotification
HolidayRoundMode	activarModoFestivo, agregarDiaFestivo	activateHolidayMode, addHoliday

ProductMultiCode	agregarCodigo, eliminarCodigo	addCode, removeCode
SlowMovingReport	analizarProductos, crearReporte	analyzeProducts, createReport
StockMinAlert	configurarMinimo, generarAlerta	configureMinimum, generateAlert
ThermalPrinting	conectarImpresora, imprimirTicket	connectPrinter, printTicket
UIAdaptive	aplicarTema, altoContraste	applyTheme, highContrast
WeightSales	reducirPeso, calcularPrecio	reduceWeight, calculatePrice

Evidence:

[illegible]

The screenshot shows a Zoom meeting interface. On the left, a window titled "UML-Enterprise - Visual Paradigm Enterprise (Evaluation Copy)" displays a UML diagram. The diagram features a central actor connected to two large rectangular boxes, each containing a list of use cases. The left box is labeled "a" and the right box is labeled "b". The actor is a stick figure with a circular head. The use cases are represented by small rectangles with a person icon. The interface includes a menu bar (Dash, Project, ITT, Agile, Diagram, View, Team, Tools, Modeling, Window, Help) and a toolbar with icons for New, Open, Save, Close, Print, Export, Import, Referenced Project, Properties, and Community Circle. A sidebar on the left lists various UML elements like Use Case, Association, Actor, System, Include, Extend, Dependency, Generalization, Collaboration, Element Group, Note, Archival, Constraint, Containment, Allocate, Package, and Diagram Overview. The main canvas shows the diagram with lines connecting the actor to the use cases in both boxes. On the right, there are four video thumbnails of participants: Ronald Tipan, Sanchez, Tipantza Alexander, and Adrian Gustavo. At the bottom, the Zoom status bar shows the time 21:47, the user ID qdv-ejyv-tip, and icons for chat, video, screen sharing, and other controls.

